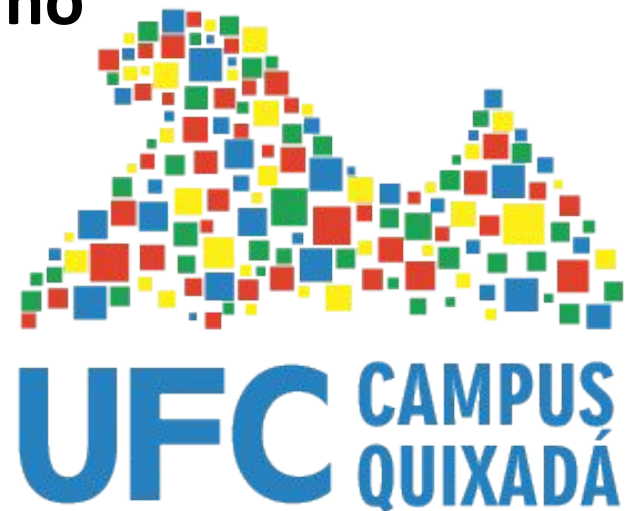


Introdução ao JavaScript

Prof. Sidartha Carvalho



AULA 1/4

JS: Introdução

O Trio do Desenvolvimento Web



HTML

Define o conteúdo das páginas web



CSS

Especifica o design das páginas web



JavaScript (JS)

Fornece "comportamento" às páginas

O Poder do JavaScript Além da Web

- Não é usado somente em páginas Web
- Presente em aplicações desktop e servidores

Exemplos de Ecossistema:

NodeJS (servidor), MongoDB, CouchDB, etc



JS: Introdução ao JavaScript

JS ≠ Java

Apesar do nome similar, surgiu em 1995 aproveitando a popularidade do Java na época como estratégia de marketing.



Origens e Padrão



- Criada por **Brendan Eich** em 1995.
- Padronizada em 1997.
- **ECMAScript** é o nome oficial.

JS no Contexto Web

- Manipular componentes **HTML/CSS**.
- Alterar estilos visualmente.
- Adicionar **comportamento** e eventos.



JS: alterando HTML

```
<!DOCTYPE html>
<html>
<body>

<h2>O que posso fazer com JavaScript?</h2>

<p id="meup">JavaScript pode alterar o conteúdo de um elemento HTML.</p>

<button type="button" onclick='document.getElementById("meup").innerHTML =
  "Novo texto usando JavaScript!'">Click Me!</button>

</body>
</html>
```

JS: alterando HTML

```
<!DOCTYPE html>
<html>
<body>

<p>JavaScript pode alterar os atributos dos elementos HTML.</p>

<p>Vamos alterar o endereço de uma imagem de forma dinâmica.</p>

<button onclick="document.getElementById('myImage').src='pic_bulbon.gif'">Li
gar lâmpada</button>



<button onclick="document.getElementById('myImage').src='pic_bulboff.gif'">A
pagar a lâmpada</button>

</body>
</html>
```

JS: alterando CSS

```
<!DOCTYPE html>
<html>
<body>

<p id="demo">JavaScript pode alterar o estilo de um elemento HTML.
</p>

<button type="button" onclick="document.getElementById('demo').style.fontSize='35px'">Clique aqui!</button>

</body>
</html>
```

JS: <script>: alert()

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Aula de JS</title>
  <script>
    alert("Olá, Mundo!");
  </script>
</head>
<body>
  <h1>JavaScript</h1>
  <h2>Linguagem de programação</h2>
</body>
</html>
```



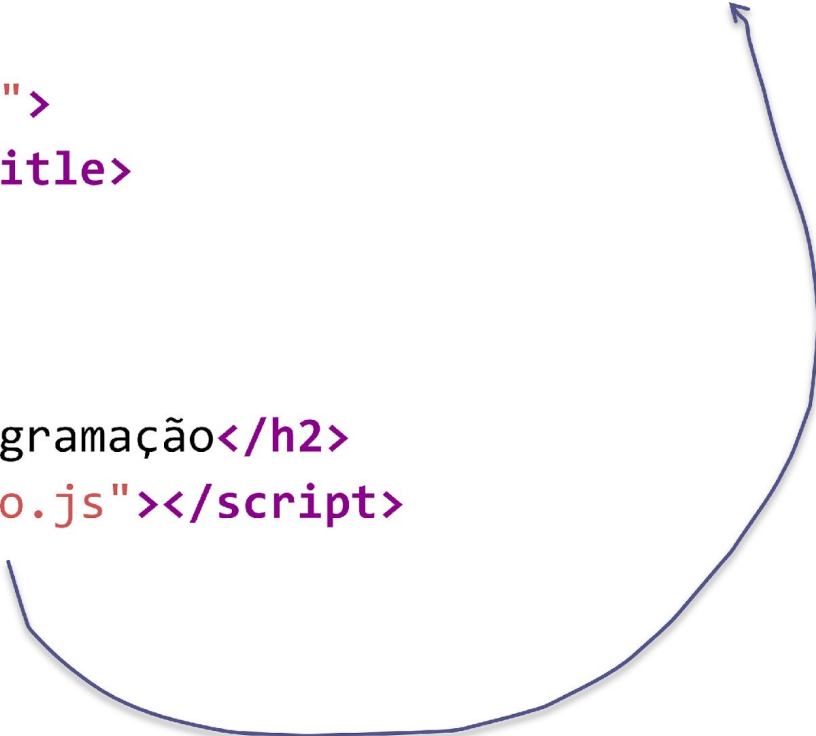
A tag <script> é usada para inserir código JS, pode ser inserida no <head>, no <body> ou em arquivo específico.

Externalizando...

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Aula de JS</title>
  </head>
  <body>
    <h1>JavaScript</h1>
    <h2>Linguagem de Programação</h2>
    <script src="js/hello.js"></script>
  </body>
</html>
```

js/hello.js

```
alert("Olá, Mundo!");
```



JS: Calculadora

```
<html>
<head><title>Calculadora</title></head>
<body>
<form id="calc">
<input id="n1" /> + <input id="n2" />
<button id="botao">OK</button>
</form>
<p id="resultado"></p>
<script>
function exibe() {
res.textContent = parseFloat(n1.value) +
parseFloat(n2.value);
return false;
}
var n1 = document.getElementById('n1');
var n2 = document.getElementById('n2');
var res = document.getElementById('resultado');
botao.onclick = exibe;
</script>
</body>
</html>
```

Estrutura HTML

Define os campos de entrada (n1, n2) e o botão de ação para coletar os dados do usuário.

Lógica JS

Captura os valores, realiza a conversão para número (parseFloat) e exibe a soma no DOM.



Manipulação de DOM

JS: console.log()

O que é?



Função usada para imprimir dados no console.

- Depuração e erros
- Acompanhamento de execução
- Informações importantes

Como usar?



Dentro de tags <script>, inclua a chamada:

```
console.log('teste')
```

Como visualizar?



Abra o **Modo Desenvolvedor** no seu navegador:

- **Chrome:** Control + Shift + C
- **Firefox:** Control + Shift + K

JavaScript: Comentários



Linha Única

Utilizado para notas curtas ou desativar código temporariamente. Inicia com duas barras inclinadas.

```
// Comentário simples
```



Múltiplas Linhas

Ideal para documentação extensa ou blocos de comentários explicativos.

```
/* Este comentário  
abrange várias  
linhas... */
```



JS: tipos de dados

Tipagem dinâmica: o tipo é atribuído a partir do valor da variável.

Dados Primitivos

- **Number:** Valores numéricos.
- **String:** Sequências de caracteres.
- **Boolean:** true ou false.
- **Undefined:** Variável sem valor.

Dados Complexos

- **Object:** Coleção de propriedades.
 - Null: Ausência intencional de valor.
- **Function:** Blocos de código executáveis.



JS: expressões



JS avalia expressões da esquerda para a direita

```
var x = 16 + 4 +  
"Volvo";  
// Resultado: "20Volvo"
```

```
var x = "Volvo" + 16 + 4;  
// Resultado: "Volvo164"
```

i Para evitar problemas, use sempre os parênteses:
`(16 + 4) + "Volvo"`

JS: tipos de dados



Undefined

```
var x;
```

Quando uma variável é declarada mas não recebe valor, seu tipo inicial no JavaScript é undefined.



Number

```
x = 5;
```

Ao atribuir um número à variável, o JavaScript dinamicamente altera seu tipo para Number.

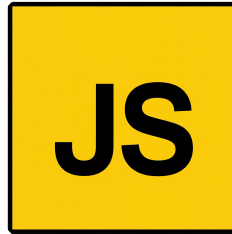


String

```
x = "John";
```

Valores entre aspas transformam o tipo da variável para String, demonstrando a tipagem dinâmica.

JS: tipos de dados



Conversão de string para número (defina a base)

```
parseInt("123",  
10)
```

123

Base Decimal

```
parseInt("010",  
10)
```

10

Base Decimal

```
parseInt("11",  
2)
```

3

Base Binária

JS: String



Comprimento

```
"hello".length
```

5



Acesso a Caractere

```
"hello".charAt(0)
```

'h'



Substituição

```
"hello,  
world".replace("hello",  
"goodbye")
```

"goodbye, world"



Transformação

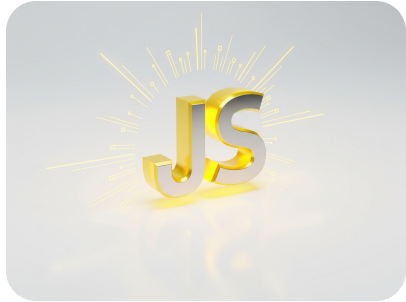
```
"hello".toUpperCase()
```

"HELLO"

Strings

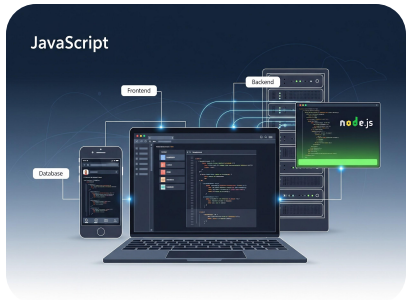
String	<code>length</code>	Returns the number of characters in a string
	<code>concat()</code>	Joins two or more strings
	<code>indexOf()</code>	Returns the position of the first occurrence of a specified string value in a string
	<code>lastIndexOf()</code>	Returns the position of the last occurrence of a specified string value, searching backward from the specified position in a string
	<code>match()</code>	Searches for a specified string value in a string
	<code>replace()</code>	Replaces some characters with others in a string
	<code>slice()</code>	Extracts a part of a string and returns the extracted part in a new string
	<code>split()</code>	Splits a string into an array of strings
	<code>substring()</code>	Extracts the characters in a string between two specified indexes
	<code>toLowerCase()</code>	Displays a string in lowercase letters
	<code>toUpperCase()</code>	Displays a string in uppercase letters

Resumo da Aula



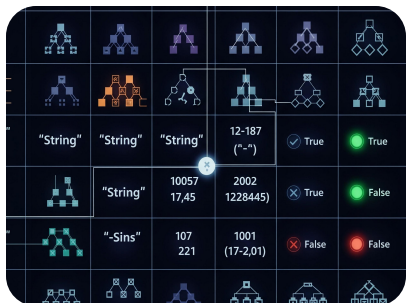
Introdução ao JavaScript

JavaScript é uma linguagem de programação versátil e essencial para a web moderna, permitindo a criação de interfaces dinâmicas e interativas que melhoram a experiência do usuário.



Usos e Aplicações

Desde o desenvolvimento front-end com frameworks modernos até aplicações no servidor com Node.js, o JavaScript está presente em quase todos os domínios da tecnologia atual.



Tipos de Dados e Expressões

Entender como manipular tipos primitivos, objetos e expressões lógicas é a base fundamental para construir algoritmos eficientes e lógicos em qualquer aplicação.

JS: Arrays

```
let cars = ["Saab", "Volvo",  
"BMW"];
```



[0] "Saab"



[1] "Volvo"



[2] "BMW"

Arrays são variáveis especiais que podem conter mais de um valor por vez. Em JavaScript, os arrays são indexados a partir de zero, permitindo organizar dados relacionados de forma eficiente e acessível.

JS: Objects

An **object** is a collection of related data and/or functionality. These usually consist of several variables and functions, which are called properties and methods when they are inside objects.

```
// Defining an Object
```

```
let person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 50,  
  eyeColor: "blue"  
};  
  
console.log(person);
```

Object Structure

Key: firstName → **Value:** "John"

Key: lastName → **Value:** "Doe"

Key: age → **Value:** 50

Key: eyeColor → **Value:** "blue"

Console Output



top



Filter

Default levels

JS: variáveis

Palavra-chave let

Utilizada para declarar variáveis com escopo de bloco.

```
let a;  
let name = "simon";
```

Valor Inicial

Variáveis declaradas sem atribuição recebem o valor:

undefined



Atenção: Variáveis Globais

Omitir **let** ou **var** torna a variável global. **Evite esta prática!**

```
var mensagem = "Olá mundo";  
console.log(mensagem);
```

JS: conversão de tipos



String para Inteiro / Float

```
let textoInteiro = "10";  
let inteiro = parseInt(textoInteiro); // 10  
let float = parseFloat("10.22"); // 10.22
```



Número para String (Formatado)

```
let milNumber = 1000;  
let milString = milNumber.toFixed(2); // "1000.00"
```



Dica de Performance:

Use **parseInt()** para converter strings em números inteiros e **toFixed()** para formatar casas decimais, retornando sempre uma **String**.

JS: operadores



Numéricos: Operações matemáticas básicas (+, -, *, /) e o operador de resto (%).



Atribuição Encurtada: Combina operação e atribuição em um único passo (=, +=, -=, *=, /=, %=).



Incremento e Decremento (++ , --):

- **Prefixado (++x):** Altera o valor e depois retorna o novo valor (Ex: se x=3, retorna 4).
- **Pósfixado (x++):** Retorna o valor atual e depois altera a variável (Ex: se x=3, retorna 3 e então x vira 4).

JS: operadores



Concatenação de Strings

Une textos usando o operador `+`. `"hello" + " world"` resultará em **"hello world"**.



Coerção de Tipos

O JS converte números em strings automaticamente durante a soma se houver uma string presente:

- `"3" + 4 + 5` → **"345"** (avalia da esquerda para a direita)
- `3 + 4 + "5"` → **"75"** (soma os números antes da string)



Dica Pro:

Adicionar uma string vazia (`" "`) a qualquer valor força a sua conversão imediata para o tipo **String**.

JS: comparativos

Básicos: Para números e strings use `<`, `>`, `<=`, `>=`

Igualdade (==)

Coerção de Tipos:

Converte os valores para o mesmo tipo antes de comparar.

```
"dog" == "dog" // true  
1 == true // true
```

Flexível, mas pode causar bugs inesperados.

Identidade (===)

Rigoroso:

Compara o valor e o tipo. Sem conversão automática.

```
1 === true // false  
true === true // true
```

Recomendado para evitar erros de lógica.

JS: while e do..while

while

Testa a condição **antes** de executar o bloco. Se for falsa no início, o código nunca será executado.

```
while (true) {  
  // loop infinito!  
}
```



Pré-teste:

0 ou mais iterações.

PAREI AQUI

do..while

Executa o bloco **primeiro**, depois testa a condição. Garante ao menos uma execução.

```
do {  
  var input = get_input();  
} while (isValid(input))
```



Pós-teste:

1 ou mais iterações.

Iterações em JavaScript: O Laço for

for (Clássico)

```
for (var i = 0; i < 5; i++) {  
  // Executa 5 vezes  
}
```

for...of

```
for (const num of lista) {  
  console.log(num);  
}
```

for...in

Itera sobre as **Chaves** (nomes das propriedades ou índices).

Ideal para objetos e extração de chaves de propriedades.

for...of

Itera sobre os **Valores** (o conteúdo real dos dados).

Ideal para arrays, strings e coleções iteráveis.

JS: switch

Standard Syntax

```
switch (action) {  
  case 'draw':  
    drawit();  
    break;  
  
  case 'eat':  
    eatit();  
    break;  
  
  default:  
    donothing();  
}
```

Comparações usam Igualdade Estrita

A instrução switch usa o operador `===` para corresponder aos casos, garantindo que tanto o valor quanto o tipo devam coincidir.

Expressions are Allowed

```
switch (1 + 3) {  
  case 2 + 2:  
    yay();  
    break;  
  
  default:  
    neverhappens();  
}
```

Operadores Lógicos: && (Short-circuit)

&& (AND) e **|| (OR)** executam o segundo operando dependendo do resultado do primeiro. Isso é conhecido como avaliação de curto-circuito.

Exemplo: Checagem de Objetos

```
var name = o && o.getName();  
  
// Se 'o' for true (não null ou undefined):  
Retorna o.getName()  
  
// Se 'o' for false:  
Retorna o (false)
```

Tabela de Verdade & Retornos

1	a1 = true && true	// t && t retorna true
2	a2 = true && false	// t && f retorna false
3	a3 = false && true	// f && t retorna false
4	a4 = false && (3 == 4)	// f && f retorna false
5	a5 = 'Cat' && 'Dog'	// t && t retorna "Dog"
6	a6 = false && 'Cat'	// f && t retorna false
7	a7 = 'Cat' && false	// t && f retorna false
8	a8 = '' && false	// f && f retorna ""
9	a9 = false && ''	// f && t retorna false

PAREI AQUI

JS: Operador Lógico OU (||)

Lógica de Curto-Circuito: `a || b`

Retorna o primeiro valor **Verdadeiro (Truthy)** encontrado. Se todos forem **Falsos (Falsy)**, retorna o último valor.

Valores Falsy em JS: `false, null, NaN, 0, "", undefined`

// Casos Booleanos

```
o1 = true || true → true
o2 = false || true → true
o3 = true || false → true
o4 = false || (3==4) → false
```

// Casos de String

```
o5 = 'Cat' || 'Dog' → "Cat"
o6 = false || 'Cat' → "Cat"
```

// Casos Falsy

```
o7 = 'Cat' || false → "Cat"
o8 = '' || false → false
o9 = false || '' → ""
```

Dica: O JS para a busca assim que encontra o primeiro valor verdadeiro.

JavaScript: Operador Ternário

O operador ternário é uma forma compacta de escrever uma estrutura condicional **if...else**.

Sintaxe Geral

```
condição ? valor_se_verdadeiro : valor_se_falso
```

Exemplo Prático

```
var allowed = (age > 18) ? "yes" : "no";
```

Se Verdadeiro (?)

O código após o ponto de interrogação é executado se a condição for **true**.

Se Falso (:)

O código após os dois pontos é executado se a condição for **false**.


```
try {  
  // Código que pode falhar  
} catch (erro) {  
  // Lida com o erro  
} finally {  
  // Executa sempre  
}  
  
throw new Error("Erro!");
```



Gestão de Erros no JS

Robustez e confiabilidade no fluxo de execução do software.

O Fluxo de Exceções

TRY: Define um bloco de código para ser testado quanto a erros enquanto está sendo executado. É o ambiente vigiado onde as operações críticas ocorrem.

CATCH: Captura o erro gerado no try. Permite que o programa continue rodando graciosamente em vez de travar completamente o sistema.

FINALLY: Garante a execução de limpeza (como fechar conexões) independente do sucesso ou falha das operações anteriores.

THROW: Permite criar erros personalizados. Você define quando algo é "errado" para a lógica do seu negócio, disparando uma exceção manual.

JS: Tratamento de Exceções

Estrutura Try-Catch-Finally

O mecanismo para lidar com erros em tempo de execução:

- **try**: Bloco onde o código é testado.
- **catch**: Captura o erro para tratamento.
- **finally**: Executa sempre, ocorrendo erro ou não.

A Cláusula Throw

Você pode criar seus próprios erros personalizados:

- Pode lançar Strings, Numbers ou Objetos.
- **new Error()**: Cria um objeto de erro com mensagem e pilha de chamadas (stack trace).
- Interrompe a execução normal até encontrar um catch.

EXEMPLO

```
// Lançando e capturando
try {
    if(v < 0) throw "Negativo";
}catch (err){
    console.log("Erro: " + err);
} finally {
    console.log("Fim do teste");
}
```

JSON

JavaScript Object Notation

Usado para serializar objetos em formato legível ao ser humano.

```
{ "Books":  
  [  
    { "ISBN":"ISBN-0-13-713526-2",  
      "Price":85,  
      "Edition":3,  
      "Title":"A First Course in Database Systems",  
      "Authors":[ {"First_Name":"Jeffrey", "Last_Name":"Ullman"},  
                   {"First_Name":"Jennifer", "Last_Name":"Widom"} ] }  
    ,  
    { "ISBN":"ISBN-0-13-815504-6",  
      "Price":100,  
      "Remark":"Buy this book bundled with 'A First Course' - a great deal!",  
      "Title":"Database Systems:The Complete Book",  
      "Authors":[ {"First_Name":"Hector", "Last_Name":"Garcia-Molina"},  
                   {"First_Name":"Jeffrey", "Last_Name":"Ullman"},  
                   {"First_Name":"Jennifer", "Last_Name":"Widom"} ] }  
  ]  
}
```

JSON: Estrutura de Dados

Valores Base

- Tipos Primitivos:

Número, string, booleano, null

- Objetos { }:

Conjuntos de pares chave-valor

- Arrays []:

Listas ordenadas de valores

Exemplo Prático

```
{
  "alunos": [
    { "nome": "João", "notas": [8, 9, 7] },
    { "nome": "Maria", "notas": [8, 10, 7] },
    { "nome": "Pedro", "notas": [10, 10, 9] }
  ]
}
```

Por que usar JSON?

O formato JavaScript Object Notation é leve, de fácil leitura para humanos e simples de processar para máquinas, tornando-se o padrão ouro para troca de dados na web.

JS: Acesso aos Atributos

Dot Notation (.)

```
obj.name = "Simon";  
var name = obj.name;
```

Bracket Notation ([])

```
obj["name"] = "Simon";  
var name = obj["name"];
```

Ambas as formas são equivalentes

- A notação de colchetes utiliza **strings**.
- Permite acesso dinâmico em **tempo de execução** e uso de **palavras reservadas**.

Exemplo de Necessidade: Atributos Dinâmicos ou Reservados

```
var property = "first-name";  
obj[property] = "Simon"; // Única forma para chaves com hífen ou variáveis  
obj["class"] = "A1"; // Útil para palavras reservadas
```

JS: Acesso Seguro a Atributos

```
// Definição do Objeto
const usuario = {
  nome: "Ana",
  preferencias: {
    tema: "dark"
  }
};

// Acesso com Optional Chaining (?.)
// E Nullish Coalescing (??)
const rua = usuario?.endereco?.rua ??
"N/A";

// Resultado
console.log(rua); // "N/A"
```

Encadeamento Opcional (?.)

Permite ler o valor de uma propriedade sem validar cada referência na cadeia.

Se for `null` ou `undefined`, retorna `undefined` sem erro.

Coalescência Nula (??)

Retorna o operando direito quando o esquerdo é `null` ou `undefined`.

Ideal para definir valores padrão de forma segura.

JS: Sintaxe de Objetos

```
var obj = {  
  name: "Carrot",  
  "for": "Max",  
  details: {  
    color: "orange",  
    size: 12  
  }  
};
```

Acesso por Ponto (Dot Notation)

```
obj.details.color
```

```
// Retorna: "orange"
```

Acesso por Colchetes (Bracket Notation)

```
obj["details"]["size"]
```

```
// Retorna: 12
```

JavaScript: Iteração de Objetos

Em JavaScript, podemos percorrer todas as chaves de um objeto utilizando a estrutura **for ... in**.

```
var obj = { 'name': 'Simon', 'age': 25 };  
for (var attr in obj) {  
  console.log(attr + ' = ' + obj[attr]);  
}
```

Dica Importante:

A variável **attr** assume o nome de cada propriedade (chave) do objeto em cada iteração, permitindo acessar o valor correspondente via **obj[attr]**.

JS: arrays

Tipo especial de objeto onde as chaves são números e não strings.

```
// Constructor syntax
```

```
var a = new Array();
```

```
a[0] = "dog";
```

```
a[1] = "cat";
```

```
a[2] = "hen";
```

```
a.length // 3
```

```
// Literal syntax:
```

```
var a = ['dog', 'cat', 'hen']
```

```
// Adding elements:
```

```
var palavras = ["UFC",  
"Ensino"];
```

```
palavras.push("Inovação");
```

```
// add nova palavra ao array
```

```
// Accessing:
```

```
palavras[0] // "UFC"
```

```
palavras[2] // "Inovação"
```

JS: Arrays Dinâmicos e Esparsos

Em JavaScript, arrays crescem automaticamente ao atribuir valores a índices altos, preenchendo o intervalo com "buracos" (empty slots).

REPRESENTAÇÃO DO ARRAY "A"



```
var a = ["dog", "cat", "hen"];  
// length: 3  
  
a[100] = "fox";  
// Expande o array para 101 itens  
  
a.length; // 101  
typeof(a[90]); // "undefined"
```

Conceito Chave:

Ao definir um índice distante, o JS reserva o espaço, mas não inicializa os intermediários. **Índices vazios não existem fisicamente na memória até serem atribuídos.**

Boa Prática (Append Seguro):

```
a[a.length] = item;
```

Adiciona ao final sem deixar buracos.

JS: arrays

length, concat(), join()

O atributo **length** retorna o número de elementos. **concat()** une arrays e **join()** converte os membros em uma string única com delimitador.

pop() & push()

pop() remove e retorna o último elemento do array. **push()** adiciona novos membros ao final do array e retorna o novo comprimento total.

shift() & unshift()

shift() remove o primeiro elemento da lista. Já o **unshift()** adiciona membros ao início do array, deslocando os índices existentes.

reverse() & sort()

reverse() inverte a ordem dos elementos. **sort()** organiza os elementos do array (por padrão, em ordem alfabética ou crescente).

slice() & splice()

slice() seleciona parte do array sem modificá-lo. **splice()** adiciona ou remove elementos em qualquer posição específica.

Resumo de Operações

Estes métodos fundamentais do JavaScript permitem a manipulação dinâmica de coleções de dados de forma eficiente e programática.

JS: funções de manipulação de arrays



Map

Cria um novo array transformando cada elemento do original.

```
[1, 2, 3] → [2, 4, 6]
```



Filter

Filtra elementos com base em uma condição, retornando apenas os verdadeiros.

```
[1, 2, 3] → [2, 3]
```



Reduce

Reduz o array a um único valor acumulado (como uma soma).

```
[1, 2, 3] → 6
```

JS: Array Manipulation

.map() method

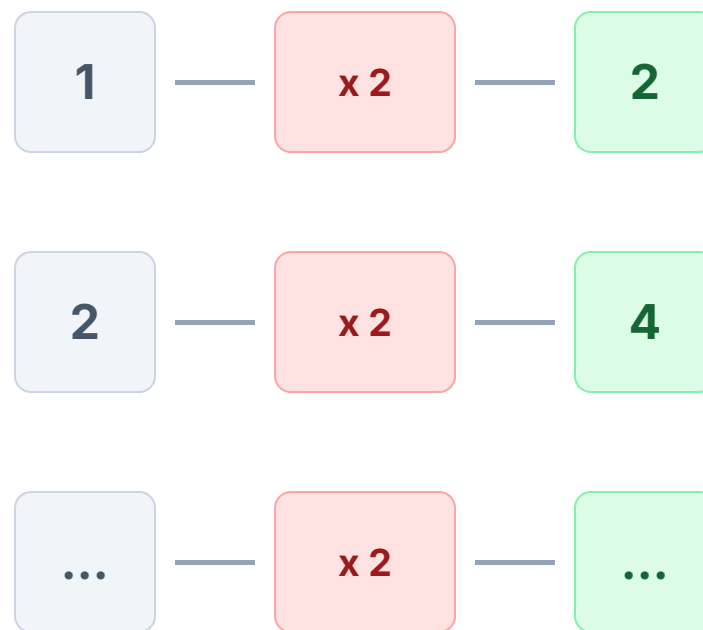
Cria um novo array preenchido com os resultados da chamada de uma função fornecida em cada elemento do array de chamada.

```
// Original Array
const numbers = [1, 2, 3, 4];

// Transformation
const doubled = numbers.map(n => n * 2);

// Result
console.log(doubled);
// [2, 4, 6, 8]
```

Visual Flow



JS: Array Manipulation

.filter()

Cria um novo array com elementos que passam em um teste específico.

```
const numbers = [1, 2, 3, 4];  
const evens = numbers.filter(item => item % 2 === 0);  
console.log(evens); // [2, 4]
```

FLUXO DE EXECUÇÃO



JS: Funções de Manipulação de Arrays

O Método `.reduce()`

O método **reduce** é utilizado para reduzir um array a um único valor. Ele executa uma função de callback em cada elemento, passando o resultado da computação anterior para o próximo passo. É ideal para somas, médias ou transformar arrays em objetos.

Arrow Function (Conciso)

```
const nums = [1, 2, 3, 4];  
const soma = nums.reduce((acc, val) => acc + val, 0);  
// Muito mais legível e eficiente visualmente.
```

JS: Funções de Manipulação de Arrays

Transformando Array em Objeto

O método `.reduce()` é extremamente versátil, permitindo converter uma lista de elementos em um objeto estruturado, utilizando um objeto vazio como valor inicial.

Exemplo: Mapeando Ids para Nomes

```
const users = [  
  { id: 1, name: 'Alice' },  
  { id: 2, name: 'Bob' }  
];  
  
const userMap = users.reduce((acc, user) => {  
  acc[user.id] = user.name;  
  return acc;  
}, {});  
  
// Resultado: { 1: 'Alice', 2: 'Bob' }
```


JS: Destructuring Arrays

Forma resumida de acessar elementos de um array

Abordagem Comum

```
const winner = scores[0];  
const secondwinner =  
scores[1];
```

Com Destructuring

```
const [gold, silver] = scores;  
const [gold, silver,  
...outros] = scores;
```

```
const scores = [90, 85, 80, 75, 70];  
  
console.log(gold); // Output: 90  
console.log(silver); // Output: 85  
console.log(outros); // Output: [80, 75, 70]
```

JS: Destructuring Objects

Acesso Tradicional

```
const nome = user.nome;  
const idade = user.idade;
```

Forma Resumida (Destructuring)

```
const { nome, idade } = user;
```

- Vincula automaticamente os atributos do objeto às variáveis com os mesmos nomes.

Renomeando Variáveis

```
const { nome: primeiroNome } = user;
```

- Extrai `user.nome` e atribui o valor à nova variável `primeiroNome`.

JS: Destructuring Objects



1. Valores Default

Se um atributo estiver ausente em alguns objetos, você pode definir um valor padrão diretamente na desestruturação.

```
const {nome, idade, filhos = 0}
```



2. Desestruturação em Parâmetros

Evita a necessidade de acessar propriedades manualmente dentro da função, tornando o código mais limpo e legível.

```
function fullName({firstName, lastName})  
{  
  console.log(firstName);  
  console.log(lastName);  
}  
  
const user = { firstName: 'Siddhartha',  
  ... };  
fullName(user);
```

Resumo:

arrays, objetos, variáveis, conversão de tipos, operadores e comparativos, JSON.